

Ai102

A local-first AI presentation generator — Ollama-only, no cloud LLM required, no login wall.

Repository	github.com/Suppakris/Ai102
Live deployment	ai102-psi.vercel.app
Based on	ALLWEONE's presentation-ai (fork, stripped for coursework)
License	MIT
Framework	Next.js 16 · React 19 · TypeScript

Table of Contents

- 1. Project Overview
- 2. Documentation vs. Actual Code — Discrepancies Found
- 3. What's Different From Upstream
- 4. Features
- 5. Tech Stack
- 6. Getting Started
- 7. Environment Variables
- 8. Database Setup
- 9. Running on Docker
- 10. Team Workflow
- 11. Usage Guide
- 12. Project Structure
- 13. Known Issues
- 14. Contributing
- 15. License

1. Project Overview

Ai102 is a local-first AI presentation generator. It is a customized, Ollama-only build derived from ALLWEONE's open-source `presentation-ai` project, stripped down for a college project. The defining idea is that text generation runs entirely on a local Ollama model — no cloud LLM subscription and no login wall stand between a user and creating a deck.

The app takes a topic, generates an editable outline, then produces a full slide deck with AI-generated images, editable themes, rich text, charts, and export to PowerPoint — all through a chat-driven in-editor agent as well as direct manual editing.

2. Documentation vs. Actual Code — Discrepancies Found

This repository's code changes fast, and its written docs (`README.md` , `CONTRIBUTING.md` , `docker-compose.yml` comments) don't always keep up. Before trusting any claim below as a working feature, this manual was cross-checked directly against the source in `src/` , `prisma/schema.prisma` , `src/env.js` , and `.env.example` . Three real discrepancies were found:

1. The OpenRouter / `LLM_PROVIDER` path in Docker is dead code

`docker-compose.yml` still wires up `LLM_PROVIDER` , `OPENROUTER_API_KEY` , `OPENROUTER_BASE_URL` , and `OPENROUTER_DEFAULT_MODEL` , and the README's Docker section describes setting `LLM_PROVIDER=openrouter` as a way to skip Ollama entirely.

Reality: none of these variables are read anywhere in `src/` — not in `src/env.js` , not in `model-picker.ts` , nowhere. The app is Ollama-only in practice, regardless of what you set these to. Treat the OpenRouter path as non-functional until someone actually wires it up in code.

2. `.env.example` 's `OPENAI_API_KEY` comment is stale

The comment above `OPENAI_API_KEY` in `.env.example` says it's "NOT required: users bring their own key in the app's Settings → 'AI Provider & API Key' (Groq's free tier works)."

Reality: there is no such Settings UI, no BYOK flow, and no Groq integration anywhere in the current codebase (confirmed by searching all `.ts` / `.tsx` files). This is leftover text from before the Ollama-only fork. The README's separate claim — that text generation is Ollama-only with all cloud providers removed — is the one that actually matches the code.

3. The "slide audit agent" is a work-in-progress, not a shipped feature

The README's Team Workflow section tells the Prompt Testing track to judge output quality "with the slide audit agent (`POST /api/presentation/audit-slide`)."

Reality: that API route does not exist anywhere under `src/app/api` . What does exist is a new `SlideAudit` model in `prisma/schema.prisma` (added roughly 13 hours before this manual was written, per the commit history), which defines fields like `score` , `pass` , `claims` , and `revisionNotes` — but there is no agent, tool, or route yet that populates it. The database shape has landed; the feature it supports hasn't.

Everything else in this manual — Next.js 16.2.1 / React 19.2.4, the `auth.ts` demo-user stub, the Ollama-only logic in `model-picker.ts` , the `pnpm` scripts, and the file paths for image generation — was verified directly against the source and matches the written docs.

3. What's Different From Upstream

This fork disables everything that requires a paid account or a login system, so it can run as a self-contained coursework build:

- **Auth is disabled.** `src/server/auth.ts` is stubbed to always return a fixed demo admin user. No Google OAuth, no `NEXTAUTH_*` variables are needed. Because the demo user's role is always `ADMIN`, every "admin-only" feature is effectively open to everyone in this build. To restore real login, revert that file and set the Google/NextAuth environment variables again.
- **Text generation is Ollama-only.** All cloud text providers (OpenAI, LM Studio, OpenRouter, Groq/BYOK) have been removed from `src/lib/model-picker.ts`. Every request resolves to a model served from `OLLAMA_BASE_URL`'s OpenAI-compatible endpoint. Legacy provider values (`openai`, `lmstudio`) from old persisted client state are caught and silently redirected to the default Ollama model instead of erroring.
- **Image generation still uses cloud providers.** FAL (Flux models) is the default and primary path (`src/app/_actions/presentation/generate-slide-image.ts`), with a Together AI path also present as a secondary/legacy provider (`src/app/_actions/image/generate.ts`).

4. Features

Core Functionality

- AI-powered outline generation, then full slide generation, running on a local Ollama model
- Editable outlines before finalizing
- Real-time slide generation
- Auto-save
- An in-editor chat agent (`presentation_agent` , built on LangGraph with Postgres-backed memory) that can edit an existing deck: change slide layout/background, replace or regenerate images, create/delete/regenerate slides, switch or create themes, and run a web search (requires `TAVILY_API_KEY`)

Design & Customization

- Multiple built-in themes (see `src/lib/presentation/themes.ts`)
- Custom theme creation
- PPTX theme import

Presentation Tools

- Present directly from the app
- Public sharing links
- PowerPoint export (`.pptx`)
- Charts, infographics, and media embeds
- Rich text editing via the Plate Editor
- Inline comments/discussions on slide content

Images

- AI image generation via FAL (Flux models — default: Flux 2 Flash; several other Flux/GPT-image models are admin-gated, which is moot since every user is admin in this build)
- Secondary image generation via Together AI (FLUX.1 family)
- Stock photos via Unsplash, Pixabay, and Giphy
- Google Custom Search image lookup (`src/app/_actions/apps/image-studio/google.ts`)

5. Tech Stack

Category	Technologies
Framework	Next.js 16, React 19, TypeScript
Styling	Tailwind CSS v4
Database	Supabase with Prisma ORM
Text Generation	Ollama (local, OpenAI-compatible endpoint), via LangChain + LangGraph agent
Image Generation	FAL (Flux models, primary), Together AI (secondary path)
UI Components	Radix UI
Text Editor	Plate Editor (<code>platejs</code>)
File Uploads	UploadThing
Lint / Format	Biome (canonical — see <code>lint</code> / <code>check</code> scripts)

6. Getting Started

Prerequisites

- Next.js 16
- pnpm (repo is pinned to `pnpm@11.1.3` ; uses `pnpm-lock.yaml` / `pnpm-workspace.yaml`)
- A PostgreSQL database (Supabase or Neon both confirmed to work)
- [Ollama](#) installed and running locally, with at least one model pulled (e.g. `ollama pull llama3.2:3b`)
- Optional provider API keys depending on which features you want (see Section 7)

Installation

1. Clone the repository

```
git clone https://github.com/Suppakris/Ai102.git
cd Ai102
```

2. Install dependencies

```
pnpm install
```

The `postinstall` script runs `prisma generate` automatically.

3. Set up environment variables

```
cp .env.example .env
```

Fill in at least `DATABASE_URL` . See Section 7 for everything else.

4. Set up the database — see Section 8.

5. Start Ollama and pull a model (if you haven't already)

```
ollama pull llama3.2:3b
```

6. Run the dev server

```
pnpm dev
```

The app runs at `http://localhost:3000` and redirects straight to `/presentation` — there's no separate marketing page and no login screen (see Section 3).

7. Environment Variables

Copy `.env.example` to `.env` and fill in what you need. `.env.example` is the source of truth; the notable variables are below.

Variable	Required?	Purpose
<code>DATABASE_URL</code>	Required	Postgres connection string, used at runtime and for migrations. For Supabase, use the Transaction pooler (port 6543) — this app runs on Vercel serverless functions, and Session/Direct connections exhaust a pooler's max-clients limit fast since they're held for the life of the client instead of released per query.
<code>FAL_API_KEY</code>	Optional	Primary AI image provider (FAL/Flux). Without it, image generation fails gracefully with a "not configured" error.
<code>OLLAMA_BASE_URL</code>	Optional	Point at a remote/tunneled Ollama instance (e.g. via ngrok) instead of localhost.
<code>OLLAMA_DEFAULT_MODEL</code>	Optional	Override the default model (<code>llama3.2:3b</code>).
<code>OLLAMA_NUM_CTX</code>	Optional	Context window in tokens (default: 8192). Ollama's own default (often 2048) is smaller than this app's generation system prompt, which makes the model see truncated instructions and produce broken decks. Lower it only if generation requests start timing out on slow/CPU-only hardware.
<code>OLLAMA_MAX_OUTPUT_TOKENS</code>	Optional	Max output tokens per generation request. Unset by default. Same trade-off as <code>OLLAMA_NUM_CTX</code> .
<code>TOGETHER_AI_API_KEY</code>	Optional	Secondary image generation path (Together AI FLUX models).
<code>UPLOADTHING_TOKEN</code>	Optional	Image storage for AI-generated images.
<code>UNSPLASH_ACCESS_KEY</code>	Optional	Stock photo search.
<code>GOOGLE_CUSTOM_SEARCH_API_KEY</code> + <code>SEARCH_ENGINE_CX</code>	Optional	Google Custom Search image lookup.
<code>TAVILY_API_KEY</code>	Optional	Web search tool for the outline generator and the in-editor chat agent.
<code>OPENAI_API_KEY</code>	Optional, unused	Declared in <code>src/env.js</code> but not read by any current code path. <code>.env.example</code> 's comment about a BYOK "AI Provider & API Key" settings screen is stale — no such screen exists. Safe to leave blank. stale doc — see Section 2

All optional integrations degrade gracefully when unset — features that need them just no-op with an error message instead of crashing. Auth-related variables (`GOOGLE_CLIENT_ID` / `GOOGLE_CLIENT_SECRET` / `NEXTAUTH_SECRET` / `NEXTAUTH_URL`) are commented out and not needed since login is stubbed.

8. Database Setup

The schema is managed with **SQL migrations** (`prisma/migrations/`). Once `DATABASE_URL` is set:

```
pnpm db:migrate:deploy # prisma migrate deploy — applies all pending migrations
pnpm db:studio         # optional: browse the DB with Prisma Studio
```

When you change `prisma/schema.prisma`, create a new migration (this also applies it to your dev database):

```
pnpm db:migrate:dev # prisma migrate dev — generates a new SQL migration from the schema diff
```

Commit the generated folder under `prisma/migrations/` — that SQL file *is* the schema change. `pnpm db:push` still exists for throwaway prototyping, but it bypasses migration history; don't use it on a shared database.

Migrating an existing `db push` database: if your DB already has the schema (created via `db:push` before migrations existed), baseline it once instead of re-running the initial migration:

```
pnpm exec prisma migrate resolve --applied 0_init
```

There's no separate seed step required to boot the app — the stubbed demo user is upserted automatically on first request.

9. Running on Docker

The whole stack (front-end + back-end + Postgres) runs on Docker with nothing installed on the host except Docker itself:

```
docker compose up --build
```

Boot order is handled for you: Postgres starts and becomes healthy → a one-shot `migrate` container runs `prisma migrate deploy` → the app starts at `http://localhost:3000`.

LLM access from inside the container

- **Ollama on the host (default):** the compose file points `OLLAMA_BASE_URL` at `host.docker.internal:11434`, which reaches the Ollama instance running on your machine.
- **OpenRouter:** create a `.env` file next to `docker-compose.yml` with `LLM_PROVIDER=openrouter` and `OPENROUTER_API_KEY=sk-or-...` — no Ollama needed at all. **documented, not implemented**

The OpenRouter option above is wired into `docker-compose.yml` but not into any application code — see Section 2, item 1. Don't rely on it.

The database is persisted in the `db-data` volume; `docker compose down -v` wipes it.

10. Team Workflow

Work is split into two independent tracks so both can move in parallel without stepping on each other:

Track 1 — Prompt Testing		Track 2 — DevOps
Owns	Prompt templates, agent behavior, model/provider selection, output quality	Docker, database migrations, environment config, deployment
Files	<code>src/ai/**</code> , <code>src/lib/presentation/*prompt*</code> , <code>src/lib/model-picker.ts</code> , <code>src/app/api/presentation/**</code>	<code>Dockerfile</code> , <code>docker-compose.yml</code> , <code>prisma/migrations/**</code> , <code>prisma/schema.prisma</code> , <code>.env.example</code> , Vercel settings
Test loop	Run the app against OpenRouter (<code>LLM_PROVIDER=openrouter</code>) for reproducible model behavior, or local Ollama for free iteration; judge output with the slide audit agent <code>not yet built — see Section 2</code>	<code>docker compose up --build</code> must boot a working app from a clean checkout; schema changes ship as SQL migrations via <code>pnpm db:migrate:dev</code>
PR scope	Prompt/agent changes only — no infra files	Infra changes only — no prompt edits

Rules of the road:

1. A schema change is a **DevOps-track** change even if a prompt PR needs it — split it out.
2. Prompt changes are judged by verification score (see the verification agent), not by eyeballing one lucky generation.
3. Both tracks branch from `main` and merge via PR; neither track pushes to `main` directly.

11. Usage Guide

1. Start the app (`pnpm dev`) and go to `http://localhost:3000` — you're dropped straight into `/presentation` as the demo admin user.
2. Create a new presentation from `/presentation/create` : describe the topic, review/edit the generated outline, then generate the full deck.
3. In the editor, use the built-in themes or create a custom one, edit slides directly, or use the in-editor chat agent to ask for changes (layout, images, new/regenerated slides, theme changes).
4. Export to `.pptx` , present directly from the browser, or generate a public share link.

12. Project Structure

High-level map of `src/` :

<code>src/</code>	
<code>├─ app/</code>	Next.js App Router routes
<code> ├─ presentation/</code>	Main app surface: create flow, generation-in-progress view, editor/viewer ([id]/)
<code> </code>	
<code> ├─ share/</code>	Public read-only share view for a presentation
<code> └─ api/</code>	Route handlers: presentation chat agent (agent/presentation/), outline/slide/image/diagram generation, stubbed auth endpoint, UploadThing's route
<code> </code>	
<code>├─ actions/</code>	Server Actions for image generation, notebook/presentation CRUD, image-studio tool
<code> </code>	
<code>├─ ai/</code>	Presentation-editing agent:
<code> ├─ agents/presentation/</code>	createAgent.ts – LangGraph agent with Supabase-backed chat memory
<code> </code>	
<code> ├─ tools/</code>	Slide/theme/image editing tools + web search
<code> └─ lib/</code>	Postgres checkpointing, pasted-content middleware
<code> </code>	
<code>├─ components/</code>	
<code> ├─ notebook/</code>	Primary implementation of the slide-outline UI, theming UI, editor plugins, image editor. ("Notebook" = a presentation project, not a separate note-taking product. Includes a small early-stage notes/ sub-mode.)
<code> </code>	
<code> </code>	
<code> └─ presentation/</code>	App-shell/viewer chrome (sidebar, edit panel, zoom/scroll, present mode) that composes pieces from components/notebook/
<code> </code>	
<code>├─ lib/</code>	
<code> ├─ model-picker.ts</code>	The Ollama-only LLM resolver (see Section 3)
<code> ├─ notebook/</code>	Data model for attaching source files to a presentation project, agent activity timeline
<code> </code>	
<code> ├─ presentation/themes.ts</code>	Built-in theme definitions
<code> └─ observability/</code>	Homegrown, console-only structured logger. No external service (Sentry/PostHog/etc.)
<code> </code>	
<code>├─ server/</code>	
<code> ├─ auth.ts</code>	The demo-user stub
<code> ├─ ai/</code>	LangChain ↔ AI SDK message conversion
<code> └─ share/</code>	Share-link authorization
<code> </code>	
<code>├─ config/, constants/</code>	Slide sizing/format presets, FAL image model catalog, infographic chart templates
<code> </code>	
<code>└─ provider/</code>	Root-level React providers (session, React Query, theme)

13. Known Issues

- **Document/RAG search is scaffolded but not implemented.** The notebook attachment model has `ragId` / `processingStatus` fields, and `PINECONE_API_KEY` is declared, but there's no actual vector-search tool wired into the agent. Don't expect "chat with your uploaded document" to work yet.
- `/auth/signin` **still renders a "Sign in with Google" button** that goes nowhere useful — a dead leftover from upstream since auth is stubbed. Harmless, but confusing if you stumble onto it.
- **"Admin-only" gating on some image models is a no-op in this build** — the demo user is always `role: ADMIN`, so those features work for everyone.
- Both Biome and Prettier are configured; Biome is canonical (it's what the `lint` / `check` scripts run).

14. Contributing

Getting Started

1. Fork the repository on GitHub
2. Clone your fork:

```
git clone git@github.com:Suppakris/Ai102.git
cd Ai102
```

3. Set up the development environment:

```
pnpm install
```

4. Create a new branch for your contribution:

```
git checkout -b feature/your-feature-name
```

Development Workflow

1. Make your changes locally
2. Follow the code style guidelines
3. Commit your changes using clear commit messages
4. Push your branch to your fork
5. Open a pull request from your fork to the main repository

Pull Request Process

1. Use a clear and descriptive title for your PR
2. Include a detailed description explaining what your changes do and why they should be included
3. Link any related issues using GitHub issue references
4. Wait for a maintainer to review your PR
5. Address any feedback from reviewers
6. Once approved, a maintainer will merge your PR

Coding Standards

- Follow the existing code style in the project
- Use meaningful variable and function names
- Write clear comments for complex logic
- Keep functions focused on a single responsibility
- Follow TypeScript best practices and maintain type safety

Commit Guidelines

Conventional commit messages are used:

Prefix	Meaning
feat:	New features
fix:	Bug fixes
docs:	Documentation changes
style:	Formatting changes
refactor:	Code refactoring
chore:	Build process or auxiliary tool changes

Issue Reporting

1. Check if it already exists in the [Issues](#) page
2. If not, create a new issue using the appropriate template

- 3. Provide detailed steps to reproduce bugs
- 4. For feature requests, explain the use case and benefits

15. License

MIT License. Originally copyright ALLWEONE Team; this fork is a derivative work created for coursework purposes.